

Supervised Trajectory analysis for a test of Optimal Foraging Theory

Students : Océane Alvarez - Benita Ibrahim - Nour Saad
Completed as part of the Master 1 program in Bioinformatics and Systems Biology

Supervisors: Alfonso Pérez Escudero and Roger Lloret Cabot

Place : (Laboratory, Institute, Research Organism etc): Centre de Recherches sur la Cognition Animale,
Centre de Biologie Intégrative, CNRS

Date : 12/01/26 to the end of may 2026

Keywords: Optimal Foraging Theory, Marginal Value Theorem, *Caenorhabditis elegans*, trajectories,
behavioral ecology



1. Context and objectives:	2
2. Materials and methods :	2
2.1. Phase 1 : Overview the dataset :	2
2.2. Consistency check :	2
2.3. Phase 3 : Analysis of Multiple Worms and Manual Annotation:	3
2.3.1 : Analysis of multiple worms	3
2.3.2 Manual annotation	4
2.4. Phase 4: Data Quality Control & Validation:	5
2.4.1. Integration of manual annotation in the trajectories files	5
2.4.2 Manual validation of automatically generated results	5
2.4.3 Analysis of duplicated frames distribution	5
2.4.4 Secondary filtering of remaining duplicated frames	6
3. Results:	7
3.1. Phase 2 and Phase 3: Dataset cleaning and multiple worm analysis:	7
3.2 Phase 3 : Threshold selection and manual annotation:	7
3.3 Phase 4 :Spatial Filter Validation and Tracking Quality Control:	8
3.3.1. Results of Annotation Integration	8
3.3.2 Validation of Automatically Generated Results	8
3.3.3 Duplicated Frames Distribution After Filtering	8
3.3.4 Results of Secondary Spatial Filtering	9
3.3.5 Objectives and Validation Strategy	9
3.3.6 Algorithmic Results and Specificity Proof	10
4. Assessment and Perspectives :	10
5. Project Management :	11
5.1. Task sharing :	11
5.2. Tools Used and AI :	11
5.3. Gantt chart:	11
6. References :	12

1. Context and objectives:

When we think about decision-making, we usually imagine animals with complex brains. In natural environments, food is often distributed in patches or specific areas. An organism must constantly decide whether it should stay in the current food patch or leave to search for another one. This decision is described by a key concept in behavioral ecology called the Marginal Value Theorem, which is part of Optimal Foraging Theory (1). This theorem predicts the optimal moment when an organism should leave a food patch in order to maximize its energy gain (2). However, despite being extensively tested in complex organisms, it remains largely unexplored in low-complexity ones. In light of this limitation, the goal of this project is to test whether the movements of *C. elegans* worms follow predictions from this theory.

The organism studied in this project is *Caenorhabditis elegans*, a tiny nematode worm widely used as a model organism in biology. Even though it has only 302 neurons, it can still perform surprisingly complex behaviors such as exploring its environment and searching for food. So this raises an interesting question: Can such a simple organism behave in an efficient way when searching for food? And more specifically, can its behavior follow predictions from ecological theories of optimal decision-making?

The long-term aim is to adapt and test Optimal Foraging Theory to simple non-visual organisms, which comprise most of the tree of life (from microbes to lower invertebrates), and are nevertheless understudied from the point of view of Optimal Foraging. To do so, the nematode *Caenorhabditis elegans* was used as a powerful model system, with a focus on one of the keystones of Optimal Foraging Theory, the Marginal Value Theorem, which recently has been adapted to simple organisms (3).

To answer this question, the host laboratory has collected a rich dataset, consisting of videos of a single worm exploring a patchy environment. There are a total of 1500 videos, covering 5 different environmental geometries with different inter-patch distances, and 3 different food patch densities.

The project involves preprocessing this trajectory dataset. All manual corrections and annotations necessary have already been performed, and the trajectories are ready for automated processing. However, some tracking errors still persist, and this project implemented and documented automated filtering criteria to eliminate or correct erroneous data, generating a highly reliable trajectory database.

Moreover, this dataset was already used to test The Marginal Value Theorem. The work will be used in future papers, where the same dataset will be used to test other questions.

2. Materials and methods :

2.1. Phase 1 : Overview of the dataset :

To respond to the biological question, the IVEP team collected a rich dataset of the exploratory behavior of *Caenorhabditis elegans* in an environment with regularly spaced food patches. It contains approximately one thousand five hundred trajectories of worms, each recorded over eight hours, under varying patch numbers and food densities. This dataset is ideally suited to test the *Marginal Value Theorem*.

The dataset is organised into a main folder named *Minipatches_light_20260116* (called Minipatches in this report), which contains many experimental subfolders. Each subfolder corresponds to one video experiment. Inside these folders, there were several types of files that includes :

1. trajectory files describing worm movement
2. food patch annotations
3. background images
4. metadata describing the experimental conditions.

2.2. Dataset consistency check :

The second step of our project was to check whether the files exist or not.

Automated tracking pipelines can occasionally fail due to technical limitations, unusable video footage, or experiments where no animal is present. These constraints result in incomplete tracking records or empty trajectory files. To manage these experimental variations across the large dataset, it is necessary to verify file integrity and check for structural presence.

To standardize the large trajectory dataset, an automated Python pipeline was developed using the *pathlib* module. The script systematically loops through all 1,365 video experiment directories to verify the

presence of core tracking and experimental files. For each directory, the system extracts three primary binary indicators (1 if present, 0 if missing):

- **Folder Path:** location of the analyzed video folder.
- **traj.csv exists?, foodpatches_reviewed.csv exists?, traj.mat exists?:** indicators showing whether the expected files were present.
- **traj.mat empty?:** verification of empty trajectory information (based on a threshold: 200bytes).
- **Diagnosis of video:** classification of the video as OK, NO WORM, or WEIRD.
- **frames with zero worms, one worm, and two or more worms:** frame-based statistics describing worm detections throughout the video.
- **proportion of frames with multiple worms:** computed value quantifying the proportion of frames containing multiple detections.
- **final classification of the video:** final label assigned to each video (0 worms, 1 worm clean, 1 worm with errors, or 2+ worms).

During the structural audit, a specific file-size verification step (traj.mat empty?) was implemented using the `os.path.getsize()` method.

A critical threshold was set at **200 bytes**:

- Any traj.mat file with a size **less than or equal to 200 bytes** is flagged as empty. This size corresponds to a file that contains only basic metadata headers but zero recorded coordinate data points for worms.
- Files **strictly greater than 200 bytes** are flagged as non-empty, meaning they contain active tracking coordinates.

By combining the file indicators and the 200-byte threshold, the pipeline automatically runs a logical test to classify each video folder into one of three operational categories:

1. **NO WORM:** Assigned if the binary file traj.mat is completely missing (`has_traj_mat == 0`) OR if the file size is under the 200-byte empty threshold (`traj_mat_empty == 1`).
2. **OK:** Assigned only if all three essential files (traj.mat, traj.csv, and foodpatches_reviewed.csv) are successfully present with full data integrity.
3. **WEIRD:** Assigned as a security flag if a valid, non-empty binary file exists, but the corresponding spreadsheet tracking files are missing. This immediately flags incomplete directories for further inspection.

To finish, all these results are exported into a CSV file that summarizes the status of each video.

2.3. Phase 3 : Analysis of Multiple Worms and Manual Annotation

2.3.1 : Analysis of multiple worms

The third step was to analyze the presence of multiple worms in the same video.

Normally, each experiment should contain only one worm but sometimes multiple detections occur due to tracking errors or real multiple individuals.

To verify this, the proportion of frames containing multiple worms was calculated with Python. This calculation was obtained by calculating the frames with zero, one and two or plus worms¹ to finally compute the proportion² based on these previous values. Calculating this proportion allows researchers to quantify the tracking difficulty for a given video.

To separate these ambiguous cases in two categories: one corresponding to the real presence of multiple individuals and the other to a single worm with tracking errors, a column ("final classification of the video") was added to our CSV file to assign for each video a label. This classification relies on two main criteria: the number of tracked individuals found in the frames, and the **proportion of duplicate frames** (frames where the system recorded multiple coordinate entries at the exact same time)³.

¹ **count_frames_simple():** This function reads a trajectory CSV file, checks that it contains a frame column, and counts how many frames contain 0 worms, 1 worm, or 2 or more worms, then returns these three values.

² **compute_prop_multiple():** This function calculates the proportion of frames containing 2 or more worms by dividing the number of frames_2plus by the total number of frames.

³ **define_category():** This function defines the category of each video (0 worms, 1 worm, 1 worm with errors, or 2 worms).

The exact rules used for this classification are defined as follows:

- If the proportion of multiple frames is under the 2% threshold, the video is considered to have only 1 worm by default.
- **0 worms:** The video is placed in this category if the tracking file is completely empty or if the total number of frames containing any worm detection is exactly 0.0.
- **1 worm clean:** The video is classified as clean single-worm data if the tracking file contains entries, but the proportion of duplicate frames is exactly 0.0. This means there is a stable single-worm tracking with no duplicated detections.
- **1 worm with errors:** The video is assigned to this category if it is a single-worm experiment but has a duplicate frame proportion strictly greater than 0.0 and below a set threshold. This indicates the presence of a single target worm along with short, low-frequency tracking artifacts (such as technical noise or body loops mimicking a second worm).
- **2+ worms:** The video is placed in this category if the proportion of duplicate frames is exceptionally high, crossing the noise threshold. This signature represents either an experiment that actually contains multiple moving worms, or a file with severe, persistent tracking errors that make the data unreliable.

However, this proportion alone was not sufficient to determine the origin of these multiple detections. A high proportion of frames with multiple worms may indicate the true presence of several individuals or the result from tracking errors. Indeed, the worm may sometimes disappear in the video because the experiment does not account for the borders of each Petri dish. As a result, the worm's ID can change, even though it is actually the same individual.

Therefore, an additional validation step was required to distinguish these two situations : a histogram⁴ of the proportion of frames containing multiple worms across all videos was computed in order to visualize the data distribution and help define an appropriate filtering threshold. This led to a manual inspection of selected videos, the selection was made by a Python script⁵ (videos under the threshold).

2.3.2 Manual annotation

The scripts `play_video.py` and `video_viewer.py`, provided by the host lab, were used to visualize each one of the videos and annotate it manually. The manual annotation was based on around fifty selected videos of selected videos. Each video was viewed using the video player, and the manual annotation was recorded in the "results_manual_annotation.csv". To facilitate this process, we used the team's duplicated Mamba environment, which already contained all the required libraries. This setup allowed us to run the `playvideo` scripts directly and visualize the trajectories efficiently. The videos were manually annotated one after another using these tools and a classification was made to separate detections into two categories :

1. NOISE : meaning it is still the same worm and the detection is incorrect.
2. WORM : meaning that it is actually a new worm.

⁴ **histogram()**: This function generates a histogram of the proportion of frames containing multiple worms across all videos and displays a threshold line to help visualize and identify a suitable filtering cutoff.

⁵ **histogramme_annotate()**: This function automatically identifies and lists videos with a proportion of frames containing multiple worms below a selected threshold (20%) in order to select candidate videos for manual annotation.

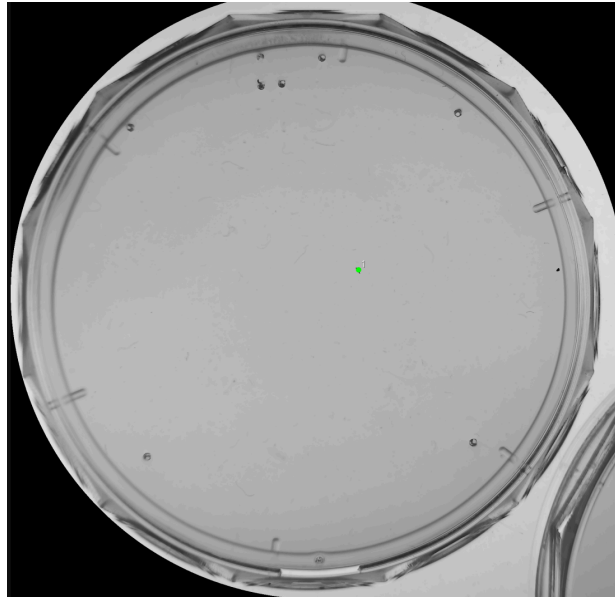


Figure 1: Capture of one of the videos showing a worm (id 1) moving in the petri box around foodpatches

After the manual annotation of the fifty videos, the annotations were saved into a CSV file (“annotation.csv”), which allowed us to correct the dataset before performing the final behavioral analysis.

For the manual annotation results_manual_annotation.csv was produced. it contains the video paths, detected individual IDs, and their assigned labels. The analysis separated ambiguous detections into two categories: WORM and NOISE. An important number of detections were identified as NOISE, showing that many apparent multiple-worm cases were actually caused by tracking errors rather than by the presence of real additional worms. These corrected annotations provided a cleaner reference dataset and were later integrated into the trajectory files to improve filtering.

2.4. Phase 4: Data Quality Control & Validation

The goal of this step is to verify whether our filtering procedure successfully removes duplicated frames and improves the reliability of the trajectories.

2.4.1. Integration of manual annotation in the trajectories files

After the manual annotation step, a Python script ⁶was developed to integrate the annotations directly into the trajectory files. The script automatically searched all traj.csv files and matched them with the “annotation.csv” table using the video path and individual IDs (id_conservative). For each video, a copy of the original trajectory file was created and enriched with a new column called manual_annotation, indicating whether each detected individual was classified as WORM or NOISE. Empty files were automatically skipped to avoid interrupting the pipeline. The resulting annotated files were saved as traj_copy.csv and served as the basis for the following filtering and quality control procedures.

2.4.2 Manual validation of automatically generated results

To verify the reliability of the automatically generated results stored in results.csv, a copy of this file, named results_manual.csv, was created. The values in selected columns (traj.csv exists?, foodpatches_reviewed.mat exists?, traj.mat exists?, traj.mat empty?, and Diagnosis of video) were removed, leaving these fields empty. Around 10 randomly selected video folders were then manually inspected and the corresponding values were filled in.

2.4.3 Analysis of duplicated frames distribution

To evaluate the quality of the tracking process and the effectiveness of filtering, two important Python scripts were developed to analyze the proportion of duplicated frames across videos:

⁶ **manual_annotation()** : The manual_annotation() function integrates manual labels into trajectory data by matching annotated individuals with their corresponding videos and creating annotated traj_copy.csv files for further filtering and analysis.

The first script⁷ recalculates the proportion of duplicated frames after filtering by removing trajectories annotated as noise and adds the resulting values as a new column (“proportion of frames with multiple worms after filtering”) to results.csv.

The second script⁸ generated histograms of duplicated-frame proportions before and after filtering for the entire dataset and separately for each video category: 0 worms, 1 worm clean, 1 worm with errors, and 2+ worms comparing the columns “proportion of frames with multiple worms” and “proportion of frames with multiple worms after filtering”.

This analysis evaluates the impact of filtering and determines whether videos expected to contain a single worm achieved a low proportion (equals to zero) of duplicated frames after processing.

Evaluating the distribution of tracking errors presented a significant visualization challenge because a large proportion of videos had a duplicated-frame proportion equal to 0.0. In particular, 1,041 videos classified as “1 worm clean” and 182 videos classified as “0 worms” contained no duplicated detections after processing. Using standard visualization methods introduced two major biases. On a linear scale, the large concentration of zero values compressed the distribution and made low-frequency tracking errors difficult to observe. On a logarithmic scale, true zero values could not be represented mathematically ($\log(0) = -\infty$), causing the cleanest videos to disappear entirely from the visualization.

To overcome this issue, an optimization strategy recommended by the supervisor was applied. A very small epsilon value ($\epsilon = 10^{-6}$) was added to all duplicated-frame proportions before applying a logarithmic scale. This adjustment preserved zero-value videos while allowing them to remain visible as a concentrated baseline peak near the 10^{-6} boundary. At the same time, the logarithmic scaling expanded the visualization of low-frequency tracking errors, improving the readability of the overall error distribution.

In addition, a dedicated boxplot representation was generated for the “1 worm with errors” category to isolate problematic videos from the rest of the dataset. This visualization allowed a clearer comparison of duplicated-frame distributions before and after filtering and highlighted the reduction in tracking-error variability after the cleaning process.

In addition, a separate boxplot was computed for the “1 worm with errors” category to specifically compare the distributions before and after filtering and better assess the effect of the cleaning process on problematic cases.

2.4.4 Secondary filtering of remaining duplicated frames

To improve videos classified as “1 worm with errors”, a secondary spatial filtering procedure was developed. Initially, 35 videos still contained duplicated detections after the first filtering step, suggesting possible tracking errors rather than the true presence of multiple worms. After the manual annotation phase, trajectories identified as noise were integrated into the trajectory files through the creation of annotated traj_copy.csv files. The secondary filtering script then analyzed the videos that still showed a non-zero proportion of duplicated frames after filtering by reading the corresponding traj_copy.csv files. For each detected individual, the script computed the average spatial position and the maximum deviation from this average position across frames.⁹

These values were compared with trajectories previously annotated as noise. A trajectory was automatically reclassified as “noise” only if its average position was spatially close to a known noise trajectory and if its maximum deviation remained below the selected threshold ¹⁰(approximately 5 pixels), indicating a stable artifact rather than a real worm trajectory. The updated annotations were saved in new traj_copy_2.csv files generated in the corresponding video folders.

3. Results

3.1. Phase 2 and Phase 3: Dataset cleaning and multiple worm analysis:

⁷ add_filtered_proportion()

⁸ generate_histograms_all_videos()

⁹ compute_individual_stats()

¹⁰ clean_remaining_noise()

The implementation of phases 2 and 3 resulted in the creation of results.csv, a summary file containing the automatically extracted information for all analyzed videos. The initial file auditing step scanned all experimental directories and extracted the exact counts for file presence and size metrics:

- **traj.csv existence¹¹:** The trajectory coordinate spreadsheet was successfully located in **1,183** videos and was missing in **182** videos.
- **foodpatches_reviewed.csv existence:** This experimental food patch file was present in **1,183** videos and missing in **182** videos.
- **traj.mat existence:** The raw binary Matlab trajectory file was present across all **1,365** videos.
- **traj.mat empty verification¹²:** The file size fell below the 200-byte threshold in **182** videos (indicating zero recorded worm data points) and was strictly greater than 200 bytes in **183** videos.

An audit of the entire dataset consisting of **1,365** processed videos revealed the following distribution across the four defined categories:

- **1 worm clean: 1,041** videos were identified. This represents the largest portion of the dataset, confirming stable single-worm tracking experiments with no duplicate frame errors.
- **0 worms: 182** videos were found. In these experiments, the tracking files were either entirely empty or contained zero worm coordinates.
- **2+ worms: 108** videos crossed the 2% threshold. These files represent experiments with multiple worms or severe tracking noise, which were successfully isolated from standard single-worm analyses.
- **1 worm with errors: 35** videos were detected. These experiments contained a single target worm but showed low-frequency tracking artifacts (under the 2% limit), pointing them out as the perfect candidates for subsequent manual annotation and secondary spatial filtering.¹³

The resulting file provided a structured overview of the entire dataset and enabled the identification of ambiguous videos requiring further inspection.

3.2 Phase 3 : Threshold selection and manual annotation

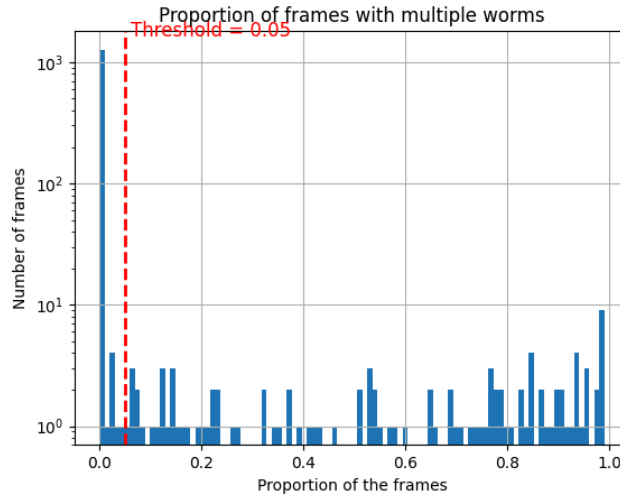


Figure 2: Histogram of the proportion of frames containing multiple worm detections.

This histogram showed that the majority of videos had very low proportions, with a large concentration of values close to zero. This indicates that most experiments contained a single correctly tracked worm with few duplicated detections. A threshold of 0.05 was selected (red dashed line) to distinguish videos with low tracking noise from potentially ambiguous cases. Videos below this threshold were considered more likely to correspond to a single worm with tracking artifacts rather than real multiple individuals and were therefore selected for manual inspection. This threshold-based selection reduced the

¹¹ `file_existence()`: helped extract the counts for traj.csv existence, foodpatches_reviewed.csv existence, and traj.mat existence.

¹² `threshold()`

¹³ `generate_histograms_all_videos()`

number of videos requiring manual annotation and allowed the analysis to focus on the most informative ambiguous cases.

3.3 Phase 4 :Spatial Filter Validation and Tracking Quality Control

3.3.1. Results of Annotation Integration

The integration process generated annotated traj_copy.csv files that served as the corrected trajectory dataset for the following quality-control procedures. Each trajectory file was enriched with manual labels describing whether detected individuals corresponded to true worms or tracking artifacts. The generated files contained:

- **id_conservative**: unique identifier associated with each detected trajectory.
- **frame**: frame number where the individual was detected.
- **x, y**: spatial coordinates of the detected individual.
- **quality**: quality information associated with the detection.
- **manual_annotation**: manually assigned label (WORM or NOISE).

These annotated trajectory files provided a corrected dataset used for filtering and subsequent trajectory analyses.

3.3.2 Validation of Automatically Generated Results

The comparison between manually verified results (that were reported in results_manual.scv) and automatically generated outputs showed strong agreement and confirmed the reliability of the detection pipeline. This validation step confirmed that the automatically generated results were sufficiently reliable.

3.3.3 Duplicated Frames Distribution After Filtering

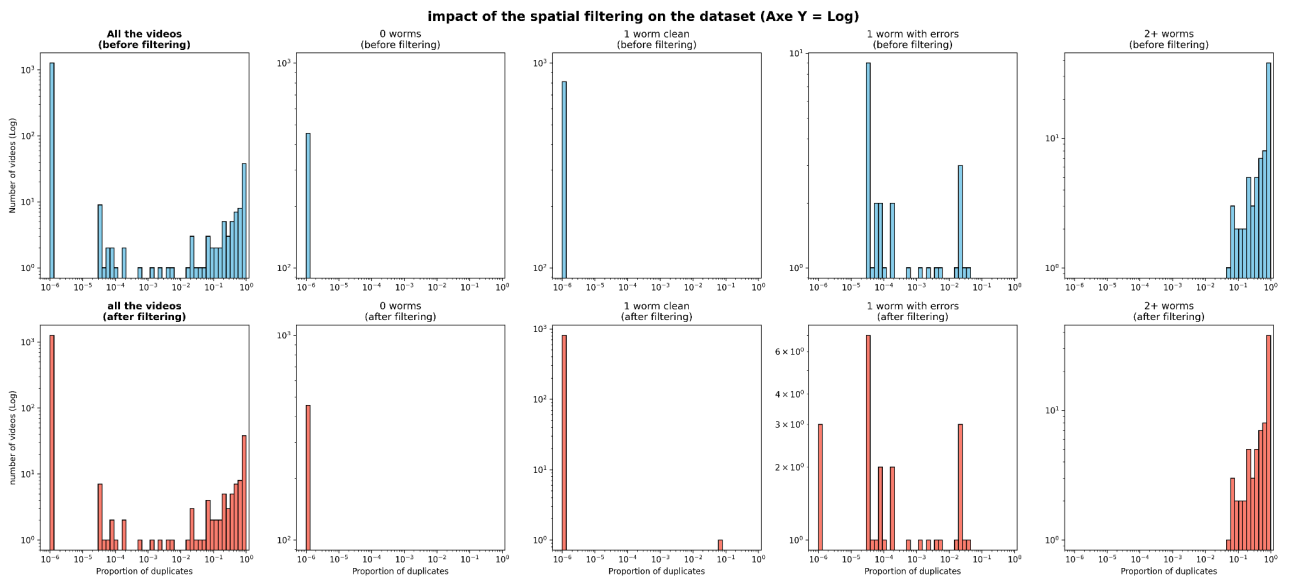


Figure 3: Histogram of duplicated-frame proportions before and after filtering across different operational categories: all videos, 0 worms, 1 worm clean, 1 worm with errors, and 2+ worms. The horizontal axis utilizes a logarithmic scale with a mathematical baseline shift ($\epsilon=10^{-6}$) to visualize true zero-value distributions.

The comparison of duplicated-frame proportions before and after filtering revealed the precise, targeted impact of the spatial data cleaning protocol (Figure 2). Globally, the vast majority of the video catalog exhibits extremely low duplication rates, represented by the massive population peak clustered at the 10^{-6} baseline. For the **0 worms** category, the distributions before (skyblue) and after (salmon) filtering are perfectly identical, confirming that the pipeline leaves already-reliable, error-free trajectories untouched. In the **1 worm clean** category, the distribution remains almost entirely unchanged at the baseline, with a negligible residual bin appearing post-filtering due to minor path alignments during the automated script execution. The targeted efficacy of the filter is most clearly visible within the **1 worm with errors** category.

Filtering reduced duplicated-frame proportions in a subset of videos classified as “**1 worm with errors**”, with several trajectories shifting toward near-zero values. However, the remaining uncleaned videos showed variable residual error levels, ranging from 10^{-4} to 10^{-1} , instead of forming one clear group. This wide dispersion is completely logical because different videos suffer from different amounts of tracking noise, some only have a few overlapping frames (placing them near 10^{-4}), while others suffer from more persistent tracking errors or worm bending (pushing them up toward 10^{-1}). Because these videos were not part of the manually annotated validation subset, they naturally retain their original, varied noise levels. In contrast, videos belonging to the **2+ worms** category maintain their high duplicated-frame proportions (extending up toward **1.0**) both before and after filtering. This behavior is biologically expected, as these experiments capture persistent coordinate entries from actual multiple individuals rather than transient technical tracking noise.

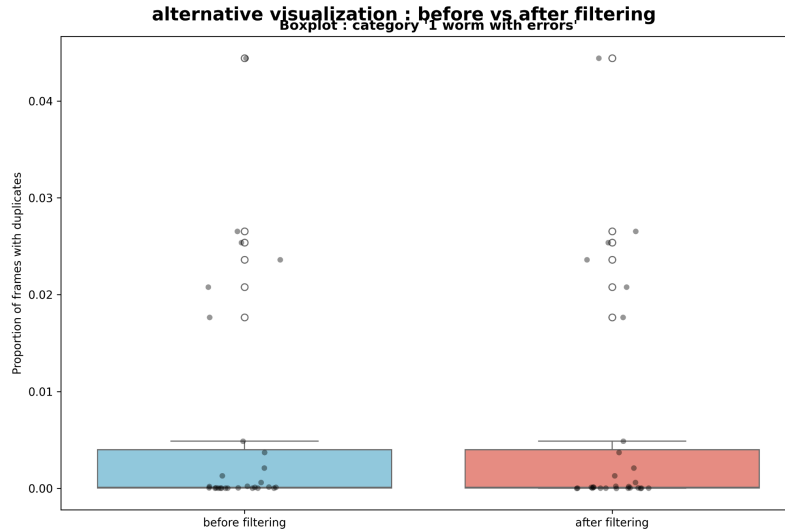


Figure 4: Boxplot comparison of “1 worm with errors” videos before and after filtering.

Figure 4 presents a boxplot comparison of videos classified as **1 worm with errors** before and after filtering. The distributions remain concentrated around low duplicated-frame values, but filtering reduces part of the variability associated with tracking artifacts. Although several outliers remained after filtering, the majority of videos showed duplicated-frame proportions close to zero, suggesting that the filtering process successfully corrected a substantial part of the tracking errors while preserving relevant trajectories.

3.3.4 Results of Secondary Spatial Filtering

The secondary spatial filtering procedure was designed to identify videos that still contained residual tracking noise after the initial validation phase. Among the **35** initially ambiguous videos classified as “1 worm with errors”, **34** still exhibited a non-zero proportion of duplicated frames after the first filtering step and were therefore selected for secondary spatial filtering. For each selected video, the algorithm computed the average spatial position and maximum deviation of each detected trajectory and compared these values with trajectories previously annotated as noise. A trajectory was reclassified as noise only if it remained spatially close to a known noise region and if its maximum deviation stayed below the selected stability threshold (approximately **5** pixels), indicating a stable artifact rather than a real worm trajectory. Following this reanalysis, only **2** videos satisfied all spatial filtering conditions and generated updated *traj_copy_2.csv* files. In the remaining videos, either no reference noise trajectories were available or the duplicated detections displayed movement patterns incompatible with stable tracking artifacts. These results suggest that most of the remaining duplicated detections were not spatially similar to previously identified noise trajectories.

3.3.5 Objectives and Validation Strategy

The primary objective of this phase was to rigorously evaluate the performance and specificity of the developed spatial filter. To validate whether the filter successfully eliminates tracking artifacts without altering high-quality biological data, a dual-scale quality control pipeline was implemented:

- **Global Dataset Audit:** Assessing the baseline tracking noise across all 1,365 processed videos.
- **Gold-Standard Validation:** Isolating the subset of manually annotated target videos (35 videos classified under the "*I worm*" categories containing tracking duplicates) to measure exact pre- and post-filtering error rates.

3.3.6 Algorithmic Results and Specificity Proof

The results suggest a high specificity of the filtering procedure.

For the **1,041** "*I worm clean*" videos and **182** "*0 worms*" videos, the pre- and post-filtering proportions remained strictly identical at 0.0 (visualized at 10–6). This proves that the spatial filter operates conservatively, ensuring that no false-positive corrections are introduced and that healthy datasets remain uncorrupted.

Within the "*I worm with errors*" cohort, the script successfully matched and cross-referenced targeted tracks. A prime example is observed in Video Row 905 (corresponding to validation Video 8), which initially exhibited a significant duplicate frame tracking artifact rate of **0.000030**. Post-filtering, the spatial filter reduced duplicated detections associated with tracking artifacts, driving the duplicate tracking rate down to **0.0** (represented as **0.000001** with epsilon).

In the global histogram (Figure 3), this effect is reflected by a distinct upward shift in the baseline clean bar of the "*I worm with errors*" sub-plot, proving that videos previously scattered across error zones were successfully cleaned and shifted into the compliant baseline pool.

4. Assessment and Perspectives :

First of all, an automatic, robust pipeline was developed to screen and audit large-scale behavioral datasets. By systematically verifying file existence and structural sizes, a raw dataset of 1,365 video directories was successfully transformed into a well-organized database with standardized quality labels. Afterward, through targeted manual annotations and secondary spatial filtering, technical tracking errors and coordinate noise were successfully identified and removed. This ensures that the final behavioral classifications (such as "*I worm clean*") are highly accurate, guaranteeing high-fidelity data integrity for downstream biological modeling. To finish, the project has now reached the operational stage where the Marginal Value Theory (MVT) can be formally tested. To complete this next step, individual patch-residence times and local food density depletion rates must be calculated from the cleaned trajectories. Once these missing parameters are extracted, they will be fitted to the analytical MVT model. With this fully validated dataset, the exact timing and mechanistic reasons driving an organism's decision to leave a food patch can be quantified, ultimately determining whether these simple organisms follow optimal foraging strategies.

During the project, the group encountered several difficulties, particularly regarding the complexity of certain Python functions and the lack of prior coding knowledge and practice. However, this initial limitation proved beneficial, as it encouraged us to explore the documentation, learn how to handle files in Python, and put newly acquired skills into practice. In addition, the project was conducted entirely in English, which made the experience both challenging and demanding. However, it also provided an opportunity to practice the language and to acquire new scientific vocabulary. Also, it is important to note that the section "*Analysis of speed*" was carried out, although the results obtained were not deemed sufficiently relevant to be included. To finish, the project could not be fully completed due to time constraints. It would have been particularly interesting to complete the final part on "*Analysis of holes*".

A possible improvement of this project would be to manually annotate all videos classified as "*I worm with errors*" in order to specifically target the most ambiguous tracking cases. Initially, 35 videos belonged to this category, but only 2 were manually annotated during this study. Among these manually inspected cases, one video was successfully corrected during the first filtering stage, while the second required the secondary spatial filtering procedure to reduce the remaining duplicated detections. This limited number of annotated problematic cases reduced the ability of the filtering pipeline to generalize to all ambiguous trajectories. In addition, the threshold selection process could be improved by automatically extracting, through a dedicated script, the duplicated-frame proportions associated specifically with the "*I*"

worm with errors” videos from the generated CSV file. Defining the filtering threshold directly from the distribution of these problematic cases would likely provide a more accurate and robust filtering strategy.

5. Project Management :

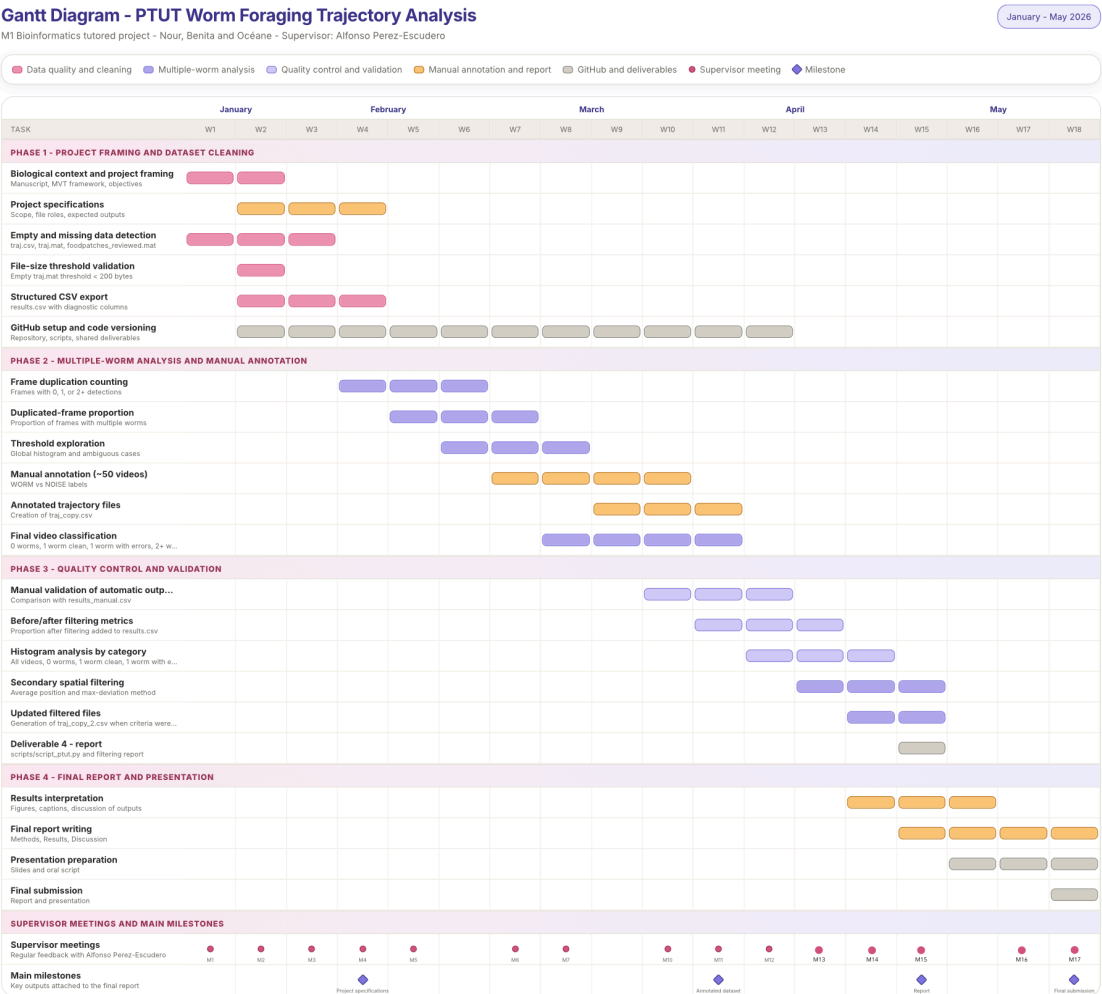
5.1. Task sharing :

During this project, all members contributed to the coding, the writing of the report and project specifications, as well as the preparation of the English oral presentation. Nevertheless, each member had a specific role: Nour was in charge of communication with the tutor and scheduling the meetings, Benita was responsible for summarizing the work and explaining the implemented code, and Océane was responsible for note-taking during each meeting. For the project, the members aimed to be as fair as possible regarding the distribution of tasks and workload.

5.2. Tools Used and AI :

The AI tools used in this project included the free version of *Copilot* to correct English mistakes in the report. In addition, *ChatGPT* was used to analyze the code, identify the types of errors, and support the development process. Finally, *Claude* was used to create the Gantt chart, allowing us to save time. Other tools were used like Wordreference, documentation for pandas.

5.3. Gantt chart:



6. Annexe:

All the deliverables are available on GitHub : [alvarezoceane/PTUT](https://github.com/alvarezoceane/PTUT). The project is centered on the preprocessing and analysis of a large trajectory dataset using Python. All manual annotations and corrections have already been performed, and the data are ready for automated processing.

7. References :

- (1) Charnov, E. L. (1976). Optimal foraging: The marginal value theorem. *Theoretical Population Biology*, 9(2), 129–136.
- (2) Nonacs, P. (2001). *State dependent behavior and the Marginal Value Theorem*. Behavioral Ecology, 12(1), 71–83.
- (3) Orjollet-Lacomme, T., Lloret-Cabot, R., Friedman, J., & Pérez-Escudero, A. (2025). *Optimal foraging for simple organisms: the single-input marginal value theory*.